



17. Hierarchical Clustering

Grouping data based on similarity

Previous Section:

 [16. Density-Based Clustering](#)

Contents:

[1. Hierarchical Clustering – The Dendrogram](#)

[2. Agglomerative Clustering](#)

[3. Divisive Clustering](#)

[4. Hierarchical Clustering – Linkage](#)

[5. Hierarchical Clustering with Python](#)

[Summary](#)

[References](#)

If K-Means is about distance, and DBSCAN is about density (with a bit of distance hidden inside its radius). Then Hierarchical Clustering is about something a little more **structural**.

It doesn't just group data. It builds a story. Not a flat answer like: "Here are your 3 clusters."

But a **process** of how everything gradually comes together, or breaks apart. And that's what makes it different.

Instead of asking: "How far are these points?" (like K-Means) or "How dense is this region?" (like DBSCAN). Hierarchical Clustering asks: ***"If I start grouping things step by step... what relationships will reveal themselves?"***

It's not just clustering. It's **clustering over time**. You don't just get clusters. You get a **history of clusters**. And that history is what we call a **Dendrogram**.

1. Hierarchical Clustering – The Dendrogram

Hierarchical Clustering is an unsupervised learning technique that organizes data into a hierarchy of clusters based on their similarity.

But let's slow down for a second, because the definition alone doesn't really capture what makes it special. Unlike most algorithms that rush straight to an answer, Hierarchical Clustering takes its time. It doesn't immediately say: "Here are your groups". Instead, it **builds them**. Step by step. Layer by layer. Almost like watching relationships form in real life.

At its core, Hierarchical Clustering doesn't force the data into a fixed number of clusters. It doesn't ask you: "How many groups do you want?". It simply starts exploring, and lets the structure reveal itself.

And as it does that, it constructs something much more powerful than just clusters: A **tree-like structure**, known as a **dendrogram**.

This dendrogram is not just a visualization. It's a story. It shows you:

- Which points were closest at the beginning
- Which clusters merged later
- And how small groups eventually became larger ones

So instead of a single answer, you get **multiple perspectives at once**. Now, there are two ways this story can unfold.

- **The first one is Agglomerative Clustering (bottom-up):** This is the more intuitive approach.
 - You start with **nothing but individuals**. Every single data point is its own cluster. Completely separate. Completely independent.
 - Then the algorithm begins asking a simple question: "**Which two are most similar?**" → It merges them. Then again and again.
 - Small clusters become bigger ones. And those clusters merge into even larger ones. → Until eventually, everything is connected.
- **The second one is Divisive Clustering (top-down):** This is the opposite perspective.

- Instead of starting with individuals, you start with **everything together**. One giant cluster.
- And then you begin breaking it apart. You split it into smaller groups. Then split those groups again.
- Over and over, until you reach individual points.

So one builds **from pieces to whole**. The other breaks **from whole to pieces**. Different directions. Same goal: **Revealing structure**.

And this is why Hierarchical Clustering is so powerful. It's not just about grouping data. It's about **understanding relationships**.

You don't just learn *what belongs together*. You learn *how strongly they belong together*, and *at what level*. That's something algorithms like K-Means simply cannot show.

Because of this, Hierarchical Clustering is often used when:

- You don't know how many clusters exist
- You care about **interpretation**, not just results
- You want to explore patterns, not just compute them

You'll see it in:

- Pattern recognition
- Customer segmentation
- Document organization
- Biological classification
- Even image grouping

Anywhere relationships matter as much as the final groups.

Now, let's make this feel real. Imagine we have twelve school subjects:

Math, Literature, Physics, English, Spanish, Geography, History, Chemistry, Biology, Computer Science, Art, and Music.

At the very beginning, everything stands alone. Twelve subjects. Twelve separate clusters. No assumptions. No grouping yet.

Then the algorithm starts observing.

- It notices something interesting: **Math, Physics, Chemistry, Biology**, and **Computer Science**. They feel similar. They all rely on logic. On structure. On analytical thinking.
 - So they merge. → A **STEM cluster** is born.
- Then it looks elsewhere: **Literature, English, Spanish, History, Geography**. These feel different. Less about numbers. More about language, culture, and human understanding.
 - So they merge into another cluster.
- And then, **Art** and **Music**: They don't quite fit with the others. They are expressive. Creative. Interpretive.
 - So they form their own group.

Now we don't have twelve clusters anymore. We have three: **STEM; Language & Social Sciences**; and **Creative Arts**.

But the process doesn't stop there. The algorithm keeps going. It starts asking: *"Which of these clusters are closer... at a higher level?"*

- Maybe **Language & Social Sciences** and **Creative Arts** merge. They both relate to human expression and interpretation.

And now we get something broader:

- **Scientific & Analytical Disciplines (STEM)**
- **Humanities & Creative Disciplines**

And if we go even further. Eventually, everything becomes one. That's the beauty of Hierarchical Clustering. It doesn't give you just one answer. It gives you **all possible answers at every level of abstraction**.

From: Very specific (Math + Physics) → To: Very general (All subjects)

And the dendrogram? That's simply the visual form of this entire journey. A map of how ideas, data points, or subjects **connect, merge, and evolve together over time**.

So if K-Means gives you a snapshot. And DBSCAN gives you density-based groups Hierarchical Clustering gives you something deeper: **Understanding**.

Now let's look at the dendrogram itself. Because everything we've talked about so far—merging, grouping, relationships— all of it becomes visible in one place.

And that place is the **dendrogram**.

The core of Hierarchical Clustering is the dendrogram. In other words, every hierarchical merging process is not just computed. It is **revealed**. Visually.

Think of it like this. Most algorithms give you an answer. Hierarchical Clustering gives you a **timeline**. And the dendrogram is that timeline drawn as a tree.

A dendrogram is a tree-like diagram that shows how data points are gradually grouped based on similarity. But more importantly, it doesn't just show the final result. It shows **every single step** along the way. From individual points, to small clusters, to larger structures. Until everything becomes one.

Now, let's break down how to actually read it. Because at first glance, it can look confusing. But once you understand the pieces, it becomes incredibly intuitive.

- **The Bottom – Where Everything Begins**

- At the very bottom of the dendrogram... you'll find the data points.
- In our example, these are the twelve subjects: **Math, Physics, Literature, Art**, and so on.
- At this stage, nothing is grouped yet. Every subject stands alone. Every point is its own cluster.

- **The Branches – Where Relationships Form**

- As you move upward, you start seeing branches. These branches represent **merges**.
- Whenever two points—or clusters—are connected, it means the algorithm has decided: **"These are similar enough to belong together."**
- The closer the connection happens to the bottom, the stronger the similarity.
- So if **Math** and **Physics** merge very early, that tells you something immediately: They are very closely related.

- **The Height – The Most Important Signal**

- Now here's the key idea—the one that makes dendrograms powerful. The **height** of each merge matters. A lot.
- It represents **distance** (or dissimilarity).
 - A **low merge** → very similar data

- A **high merge** → less similar data
- So when you see two clusters merging high up in the tree, that's the algorithm saying: ***"These don't really belong together... but I have to merge them eventually."***

And this is where Hierarchical Clustering becomes more than just clustering. It becomes **interpretation**. You're not just seeing groups. You're seeing **how natural those groups are**.

- **Cutting the Tree – Choosing Your Clusters**

- Now comes the most powerful part. Unlike K-Means, you don't decide the number of clusters at the beginning. You decide it **at the end**. And the way you do that is beautifully simple. You draw a **horizontal line** across the dendrogram.
- Where you place that line determines your clusters.
 - Cut **low** → many small clusters
 - Cut **high** → fewer, larger clusters
- Each time the line crosses a branch. That defines a cluster.

So instead of asking: "How many clusters should I choose?"

You ask something much better: ***"At what level does the data naturally separate?"***

That flexibility is what makes dendrograms so powerful. You can explore:

- Fine-grained relationships
- Broad groupings
- Everything in between

All from the same model.

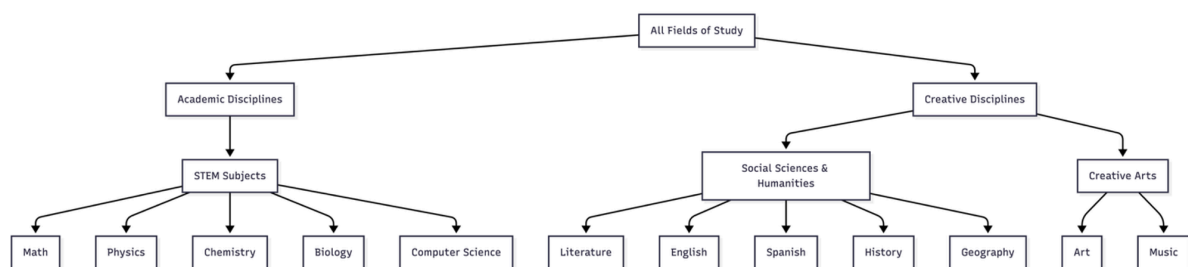
So in the end, the dendrogram is not just a visualization. It's a **map of similarity**. A way to see not just *what belongs together*, but *how strongly* and *at what level*.

And once you learn to read it. You're no longer just clustering data. You're **understanding its structure**.

Let's take that dendrogram and actually **use it**. Because up until now, we've been reading it. But the real power of Hierarchical Clustering comes from this one question: **"Where should we cut?"**

Using our example of the twelve school subjects, the answer is not fixed. It depends entirely on **where we slice the dendrogram**. And that's where things get interesting.

Here is how I would slice it:



- **Level 1 – The Top (Everything is One)**

At the very top of the dendrogram, everything is connected. All subjects merge into a single cluster. No distinction. No separation. Just one big idea

→ **Cluster 1:** All Fields of Study

This is the most general view possible. Useful? Sometimes, but often too abstract to be meaningful.

- **Level 2 – The First Separation**

Now we move slightly down. The dendrogram begins to split. And here, the algorithm might notice something fundamental:

Some subjects are more **analytical**, while others are more **creative**.

So we get:

- **Cluster 1:** Academic Disciplines
- **Cluster 2:** Creative Disciplines

Now we're starting to see structure. Still broad, but already more informative.

- **Level 3 – Clear Structure Emerges**

Go a bit lower... and things become much clearer. The algorithm now identifies more specific relationships:

- **Cluster 1:** STEM Subjects
- **Cluster 2:** Social Sciences and Humanities
- **Cluster 3:** Creative Arts

This is often where the dendrogram becomes **intuitively meaningful**. The groups make sense. They reflect how we naturally think about subjects.

- **Level 4 – The Bottom (Everything is Separate)**

At the very bottom... nothing has merged yet.

Every subject stands alone.

- Math, Physics, Chemistry, Biology, Computer Science
- Literature, English, Spanish, History, Geography
- Art, Music

Each one is its own cluster. This is the **starting point** of agglomerative clustering.

So what does this tell us? Hierarchical Clustering doesn't give you one answer. It gives you **many answers at once**.

From: Very general (everything together) → To: Very specific (everything separate) And everything in between.

The Art of Cutting - The key skill:

Choosing the **right level**.

- Cut too **high** → clusters become too broad → You lose important details
- Cut too **low** → clusters become too fragmented → You separate things that belong together

So the goal is balance. And in many real cases... The **best level** is just above the leaf nodes.

- **Leaf Nodes – Where Meaning Begins**

Let's define this clearly:

- **Leaf nodes** = individual data points (*Math, Physics, Literature, Art...*)
- The level **right above them** = first meaningful clusters

This is where the most similar items are already grouped. But clusters are not yet too general. It's the sweet spot. Clear. Intuitive. Interpretable.

From Small to Big – The Hierarchy

As we move upward:

- Small clusters → merge into larger ones
- Larger clusters → merge into broader categories

But here's the trade-off:

- Lower levels → **more detail**
- Higher levels → **more abstraction**

Too much abstraction, and you lose meaning.

A Simple Way to Think About It

Think of the dendrogram like a **family tree**.

- Individuals → leaf nodes
- Immediate family → small clusters
- Extended family → larger clusters
- Entire lineage → one big cluster

This analogy captures everything: You can study close relationships (siblings), or distant ones (ancestors) → Same data, different perspectives.

Hierarchical Clustering is powerful. But it comes with responsibility. Because the final result depends heavily on **your cut**. And that means:

- Different cuts → different conclusions
- No single "correct" answer

So choosing the right level requires:

- Careful observation
- Domain knowledge
- Sometimes additional validation

This is what makes Hierarchical Clustering special. Unlike K-Means, you don't decide the number of clusters first. You **discover it**. You explore the structure,

then choose the level that makes the most sense.

And that's the real shift in thinking: You're not forcing the data into groups. You're **reading the story it's trying to tell**.

2. Agglomerative Clustering

Agglomerative Clustering follows a **bottom-up** strategy. But instead of thinking of it as an algorithm, think of it as a process of **gradual agreement**.

At the beginning, nothing agrees with anything. Every data point stands alone. Independent. Unrelated.

And then the algorithm starts asking a very simple question: **"Which two are the most similar?"**

At the start, every subject is its own cluster: ***Math, Literature, Physics, English, Spanish, Geography, History, Chemistry, Biology, Computer Science, Art, and Music.***

Twelve subjects. Twelve separate clusters. No structure yet.

- **The First Step – Finding the Closest Pair:** The algorithm computes similarity (or distance) between **every possible pair**. And then, it picks the closest two. And merges them.

That's it. One small step. But this step repeats, again and again.

- **Early Merges – Small, Obvious Connections:** At the beginning, the merges are usually very intuitive. You can almost predict them:
 - *Art and Music* merge first → Both are creative, expressive, performance-based
 - *Math and Physics* merge → Strong analytical and mathematical foundation

These early merges happen **low in the dendrogram**, which tells us something important: These relationships are **strong and natural**

- **Building Scientific Structure.** As the process continues, clusters start growing:
 - *Chemistry joins Physics* → Shared focus on experimentation and quantitative analysis
 - *Biology joins Chemistry* → Both belong to natural sciences, similar methodologies

- *Computer Science joins Math–Physics* → Algorithms, logic, and computational theory

Now something interesting happens. We no longer just have small clusters... We have a **concept** forming: STEM Subjects

- **Meanwhile, a different story forms.** At the same time, another branch is growing. But it looks completely different.
 - *English, Spanish, and Literature* merge → Language, communication, interpretation
 - *History and Geography* merge → Human societies, context, development

Then these clusters combine. And now we see another concept emerge: Humanities and Social Sciences

- **A Separate Identity.** And then there's: *Art and Music*
They stay together. But separate from the rest. Because their nature is different. Creative. Expressive. Interpretive → Creative Arts
- **Mid-Level Structure – Clear Groupings.** At this stage, we typically have three major clusters:
 - STEM Subjects
 - Humanities and Social Sciences
 - Creative Arts

This is often the **most interpretable level**. Clear enough to understand. Detailed enough to be meaningful.

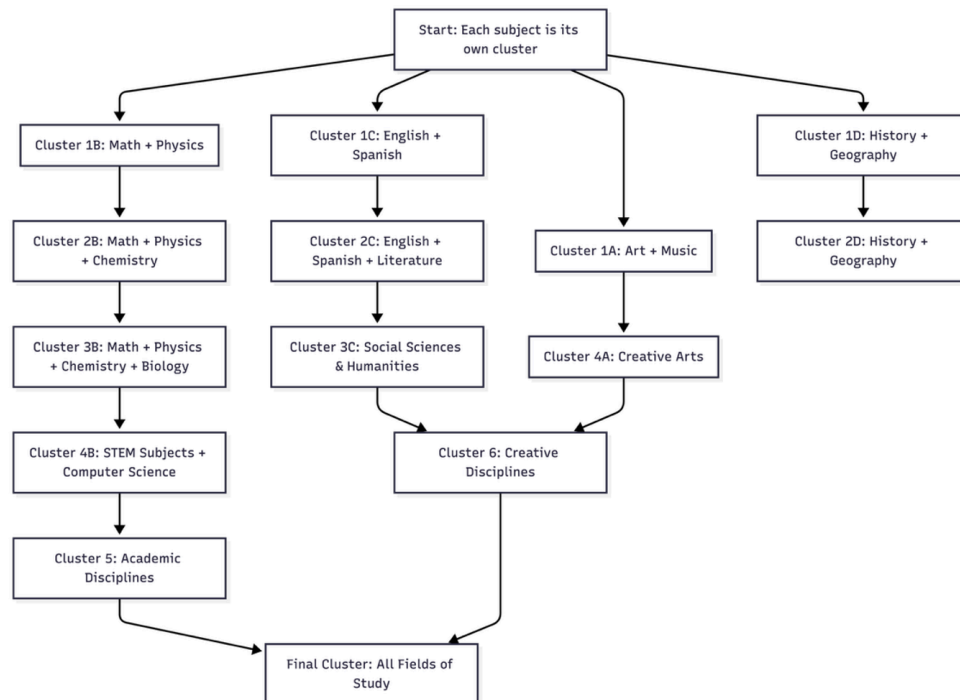
- **Higher Levels – Abstraction Begins.** But the algorithm doesn't stop. It keeps merging.
 - Now the question becomes: ***"Which clusters are closer... at a higher level?"***
 - For example: STEM + Humanities → Academic Disciplines; Creative Arts → remains separate

And eventually, everything merges into: All Fields of Study

- **The Dendrogram – A Record of Everything.** All of this—every merge, every step—is recorded in the dendrogram.

- Bottom → individual subjects
- Middle → meaningful clusters
- Top → one unified group

It's not just the result. It's the **entire journey**.



- **An Interesting Detail – Naming Clusters.** Here's something subtle, but important. Clusters don't always have clear meanings at the beginning.
 - When only two items merge, the cluster might be labeled: Cluster 1A and Cluster 2B. Because honestly, we don't know what it represents yet.
 - But as clusters grow... Something changes. Patterns start to appear.
 - *Math + Physics + Chemistry + Biology* → Now we recognize *STEM*

At this point, naming becomes meaningful.

We don't force labels early. We let them **emerge naturally**. This makes the clustering more interpretable, more intuitive, more aligned with real-world understanding.

Agglomerative Clustering is not just about merging points. It's about **building meaning over time**. Small connections form first. Then those connections grow into ideas. And those ideas evolve into structure.

And the dendrogram? It's not just a diagram. It's a **story of how similarity becomes structure**.

3. Divisive Clustering

The perspective is flipped. If Agglomerative Clustering is about **building up...** Divisive Clustering is about **breaking down**.

Divisive Clustering follows a **top-down** approach. Instead of starting with many small clusters... it starts with **everything together**. One cluster. One group. No separation.

And then it begins asking a very different question: **"Where are the biggest differences?"**

- **The Starting Point – Total Unity.** Using our twelve subjects:

Math, Literature, Physics, English, Spanish, Geography, History, Chemistry, Biology, Computer Science, Art, and Music...

At the beginning, they are all grouped into → **Cluster 1: All Fields of Study**. Everything is treated as one.

- **The First Split – The Biggest Divide.** Now the algorithm looks for the **strongest separation** in the data. Not small similarities like before... But major differences.
 - And it might find something like: Academic Disciplines **and** Creative Disciplines → This is the first cut. The most general separation.
- **Breaking Down Further.** But just like before... the process doesn't stop. Each cluster is analyzed again. And split again.
 - From Academic Disciplines, we might get: STEM Subjects → Science, mathematics, analytical reasoning
 - From Creative Disciplines, something more interesting happens. It may split into:
 - Social Sciences and Humanities → Language, culture, society

- Creative Arts → Expression, creativity, performance

Now we already see structure forming. But Divisive Clustering keeps going.

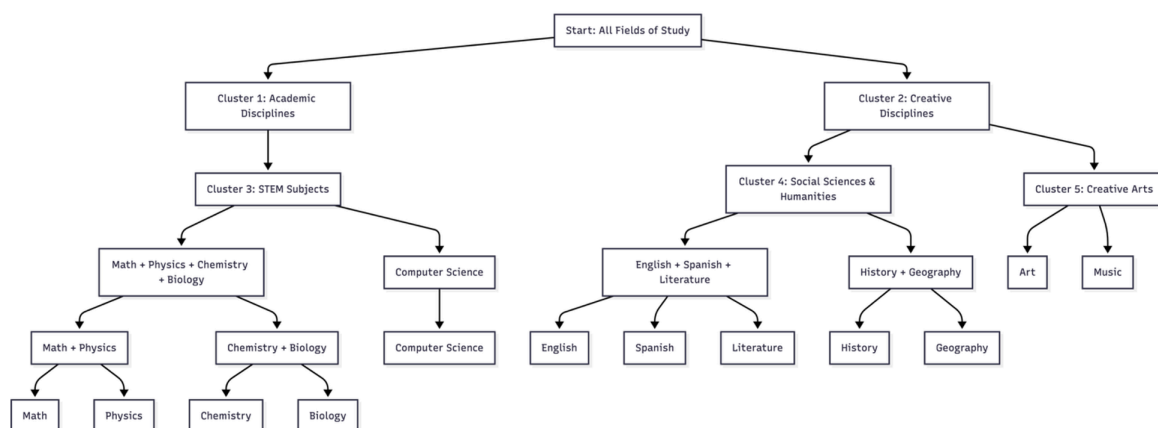
- **Moving Toward Detail.** The clusters become more specific:
 - **STEM Subjects** → *Math, Physics, Chemistry, Biology, Computer Science*
 - **Social Sciences & Humanities** → *Literature, English, Spanish, History, Geography*
 - **Creative Arts** → **Art, Music**

Each split reveals finer distinctions. Until eventually, we can split no more.

- **The Final Stage – Complete Separation.** Every subject stands alone again. Each becomes its own cluster. This is the **lowest level** of the hierarchy.

• The Dendrogram – Now in Reverse

Just like before, everything is visualized using a dendrogram.



But now, think of it differently. Instead of merging upward. You're **branching downward**.

- Top → one big cluster
- Middle → major divisions
- Bottom → individual subjects

Same structure. Opposite direction.

Two Perspectives, One Idea

At a conceptual level, Divisive and Agglomerative Clustering are mirrors of each other.

- **Agglomerative** → bottom-up → Build from individuals
- **Divisive** → top-down → Break from the whole

Different paths... Same hierarchy.

Divisive Clustering gives you something a little more unique. Instead of asking: **“What belongs together?”** It asks: **“What should be separated first?”**

This is especially useful when:

- You want to understand **major divisions** in your data
- You care about **high-level structure first**
- You want to see how broad categories split into finer ones

If Agglomerative Clustering feels like assembling a puzzle. Divisive Clustering feels like carving a sculpture. You start with a solid block. And with each cut, you reveal structure hidden inside.

And just like before, the dendrogram is not just a diagram. It's the full story of how **one idea becomes many**.

4. Hierarchical Clustering – Linkage

There's one question we've been quietly ignoring. We said the algorithm keeps merging the **closest clusters**. But what does “closest” actually mean... when a cluster contains **more than one point**?

This is where **linkage** comes in. And honestly... this is one of the most important ideas in Hierarchical Clustering.

At the beginning, everything is simple. Distance is just between two points. But after a few merges... clusters start to grow.

Now you're no longer comparing: $\text{point} \leftrightarrow \text{point}$. But: $\text{cluster} \leftrightarrow \text{cluster}$

So the algorithm needs a rule. A way to answer: **“How do we measure distance between two groups?”** → That rule is called **linkage**.

Different linkage methods answer that question differently. And because of that, they can produce **completely different clustering results** from the same

dataset.

- **Single Linkage (Minimum Distance)** - This is the simplest idea. Two clusters are considered close if **any two points (one from each cluster) are close**. So we only look at the **closest pair**.

Think of it like this: If one student from Group A is very similar to one student from Group B, the algorithm says: *"Good enough. Merge them."*

Result:

- Clusters grow like **chains**
- Long, stretched shapes
- Points can be loosely connected through neighbors

→ This is called the **chaining effect**

- **Complete Linkage (Maximum Distance)** - Now we flip the idea. Instead of the closest points. We look at the **farthest pair**. Two clusters are close only if **all points are close**

So the algorithm becomes stricter: *"If even one pair is far apart... maybe these clusters shouldn't merge yet."*

Result:

- Clusters are **compact and tight**
- More balanced shapes
- Sensitive to outliers (because far points matter a lot)

- **Average Linkage (Group Average)** - This one takes a middle ground. Instead of focusing on extremes, it computes the **average distance** between all pairs of points.

So it asks: *"On average... how similar are these two clusters?"*

Result:

- Balanced clusters
- Less extreme than single or complete
- Often reflects the overall structure well

- **Ward's Method (The Most Practical One)**

Now this one is different. It doesn't think in terms of pairwise distances directly. Instead, it asks: ***"If I merge these clusters... how much does the variance increase?"***

More specifically: It tries to **minimize the increase in within-cluster variance**. So at every step, it chooses the merge that keeps clusters compact, stable, and well-separated

Result:

- Clusters are **clean and structured**
- Similar sizes
- Often the most interpretable

→ That's why Ward's method is the most commonly used in practice.

Here's the important part. The dataset doesn't change. But the **linkage method does**. And that alone can completely change:

- The shape of clusters
- The merging order
- The final interpretation

For example:

- **Single Linkage** → long, chain-like clusters
- **Complete Linkage** → tight, compact clusters
- **Average Linkage** → balanced clusters
- **Ward's Method** → structured, stable clusters

Same data. Different stories.

Hierarchical Clustering is not just one algorithm. It's a **framework**. And linkage is the rule that guides its decisions.

So when you choose a linkage method. You're not just choosing how to measure distance. You're choosing: ***"What does similarity mean... for this problem?"***

And that choice is what ultimately shapes the entire dendrogram.

5. Hierarchical Clustering with Python

The implementation of Hierarchical Clustering is straightforward with the help of the Scikit-Learn library

- Define a function to implement Agglomerative Clustering using Scikit-Learn

```
from sklearn.cluster import AgglomerativeClustering

def agglomerative_clustering(X):
    # Agglomerative Clustering: Using Bottom-Up approach.
    clustering = AgglomerativeClustering(n_clusters=3)
    labels = clustering.fit_predict(X)
    model = AgglomerativeClustering(distance_threshold=0, n_clusters=None)
    model.fit(X)

    return labels
```

- Define another function to implement Divisive Clustering using Scikit-Learn. This is done by utilizing the K-Means algorithm to recursively split a large cluster into smaller clusters until the desired hierarchy or stopping condition is reached.

```
from sklearn.cluster import KMeans

def divisive_clustering(X, max_clusters=3):
    # Divisive Clustering: Top-Down
    clusters = [X]
    while len(clusters) < max_clusters:
        # choose the largest cluster to split
        cluster_to_split = max(clusters, key=lambda x: len(x))
        clusters.remove(cluster_to_split)

        kmeans = KMeans(n_clusters=2, random_state=42)
        labels = kmeans.fit_predict(cluster_to_split)
```

```

cluster1 = cluster_to_split[labels == 0]
cluster2 = cluster_to_split[labels == 1]
clusters.extend([cluster1, cluster2])

```

```

return clusters

```

- Execute the function + Visualization

```

import pandas as pd
from sklearn.datasets import load_iris

# Execute the function
iris = load_iris()
df = pd.DataFrame(iris.data, columns=iris.feature_names)
X = df.values
agglomerative_clustering(X)
divisive_clustering(X)

```

```

# Plot dendrograms for both hierarchical clustering models
from scipy.cluster.hierarchy import dendrogram, linkage
import matplotlib.pyplot as plt

```

```

def plot_dendrogram(X):
    plt.figure(figsize=(12, 5))
    # Agglomerative dendrogram
    plt.subplot(1, 2, 1)
    Z = linkage(X, method='ward')
    dendrogram(Z)

    plt.title("Agglomerative Clustering Dendrogram")
    plt.xlabel("Data Points")
    plt.ylabel("Distance")

    # Divisive approximation dendrogram
    plt.subplot(1, 2, 2)
    Z_div = linkage(X, method='ward')

```

```

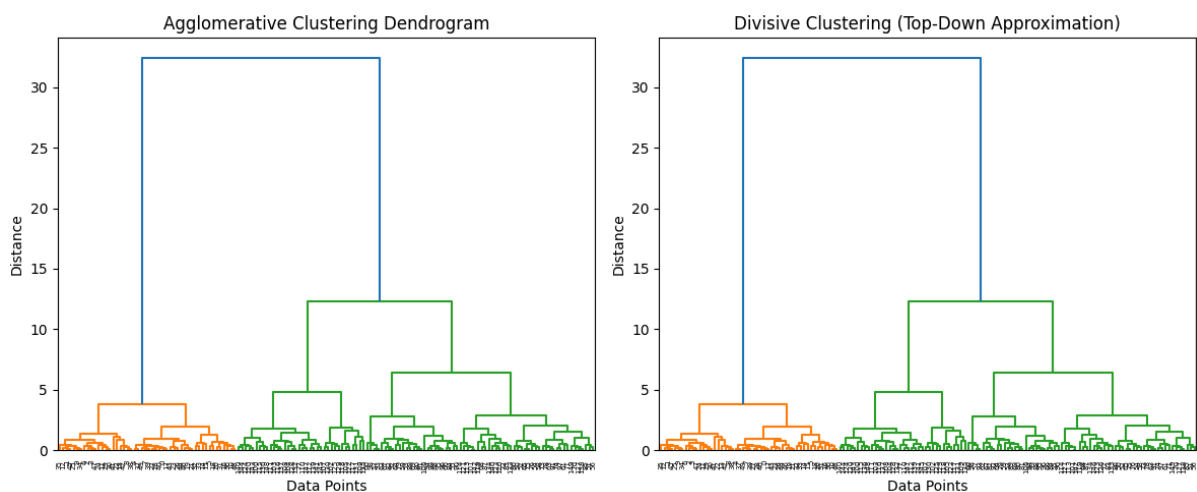
dendrogram(Z_div)

plt.title("Divisive Clustering (Top-Down Approximation)")
plt.xlabel("Data Points")
plt.ylabel("Distance")

plt.tight_layout()
plt.show()

plot_dendrogram(X)

```



Summary

Hierarchical Clustering sounds simple on the surface. But it becomes incredibly powerful the moment you realize: it's not just about grouping data. It's about **understanding structure**.

If K-Means gives you fixed clusters, and DBSCAN gives you density-based groups. Hierarchical Clustering gives you something deeper: A **hierarchy of relationships**

Instead of forcing the data into a predefined number of clusters, it lets the structure **emerge naturally**. You don't just get one answer. You get **all possible groupings**—from the most detailed to the most general.

Through the dendrogram, you can:

- See how clusters form step by step

- Understand how strongly data points are related
- Choose the level of detail that best fits your problem

And with concepts like:

- **Agglomerative (bottom-up)** → building structure
- **Divisive (top-down)** → breaking structure
- **Linkage methods** → defining similarity

You gain full control over how that structure is revealed.

So in the end, Hierarchical Clustering is not about asking: **"What are the clusters?"** It's about asking: ***"How is this data organized... at every level?"***

And that's what makes it so powerful. Not just clustering data. But **understanding how everything connects;**

References

<https://www.geeksforgeeks.org/machine-learning/hierarchical-clustering/>

<https://www.datacamp.com/tutorial/hierarchical-clustering>

<https://www.ibm.com/think/topics/hierarchical-clustering>

<https://www.mathworks.com/help/stats/hierarchical-clustering.html>

<https://www.youtube.com/watch?v=7xHsRkOdVwo>

<https://www.youtube.com/watch?v=8QCBI-xdeZI>

Next Section:

 18. Naive Bayes Classifier